

Lesson 1: Coding objectives and philosophy

Introduktion til Sundhedsdata

Emily Johnson

2026-09-01

Coding objectives of the course

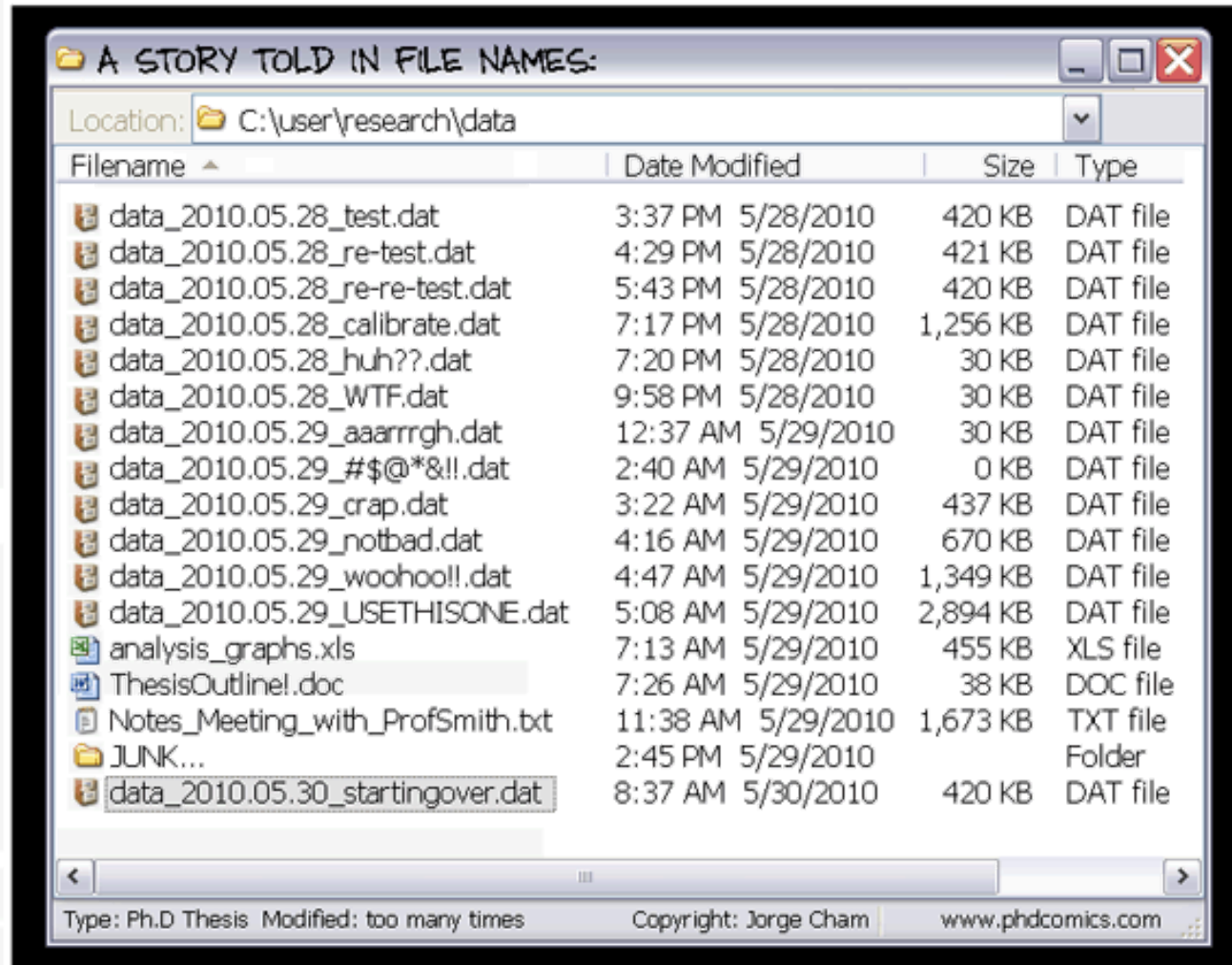
Goal: prepare you for successful data analysis work – beyond register-based research

- Good data analysts aren't just good modelers
- Technical and practical skills
- This course: coding basics, project maintenance, and how to use resources

Logistics

- **Language:** R, written and run in Positron
- **Format:** scripts are Quarto (`.qmd`) files
- **First half of demos:** basic data management
- **Second half of demos:** plots, tables, and reproducible scripts

What can go wrong ...



Jorge Cham, *PHD Comics* — phdcomics.com

Characteristics of good data analysis

- It is **clear** and easy to understand
- Results are **reproducible**
- Code is written to be **flexible** for future iterations
- Analysis is **efficient**, both in how it's written and how it runs
- Outputs are **verified** to do what they're supposed to

Would a new collaborator (or you in 6 months) know how to reproduce this analysis?

Data project checklist

Good setup

Good documentation

Good code

Data project checklist

Good setup

Good documentation

Good code

- Project organization:
 - Folders and file names
- Code versioning: Git
- R code syntax: Air

Data project checklist

Good setup

Good documentation

Good code

- Thorough code commenting
- Data dictionaries
- READMEs

Data project checklist

Good setup

Good documentation

Good code

- File paths at top of script
- Analysis files render successfully
- Functions and loops: flexibility
- Efficiency: parquet and arrow

Why focus on this?



r/PhD • 4mo ago
Both-Mushroom1847



Disorganized research workflow. How did you fix it?

Hey all,

I'm an early-stage PhD student in Econ (mostly empirical work with a bit of theory too), and I'm starting to realize that my entire research workflow is kind of broken. I feel like I'm wasting mental energy on organization instead of actual research.

Everything is scattered, papers, datasets, code, notes, across random folders, Downloads, Desktop, Google Drive, Overleaf projects, sometimes even different versions of the same thing in multiple places. I keep re-downloading papers because I can't find them. Same for data or code I know I used before.

My laptop storage fills up fast, but more than that, I feel like this chaos is slowing down my thinking and progress. I spend way too much time just looking for things or reconstructing stuff I already did.

The worst part is that I'm aware of it, but I keep telling myself "I'll organize it later" and I never do, so it just keeps compounding.

At this point I'm wondering if I should just stop everything for a few days or a week and rebuild a clean system from scratch.

So I'm curious:

Did you go through this early in your PhD?

What actually worked for you to fix it, not just in theory but in practice?

How do you organize papers vs code vs data vs writing?

Any tools that genuinely made a difference and that you could please suggest will be greatly appreciated.

Would really appreciate any concrete systems or habits that actually worked for you, especially from people doing empirical work where everything is interconnected.

Thanks!

Good setup – Project organization

Every analysis should live in its own folder with a predictable structure:

```
project/
├── README.md
├── code/
│   ├── _functions.R
│   ├── 0_read_data.qmd
│   ├── 1_clean_socioeconomics.qmd
│   └── 2_build_analysis_file.qmd
├── data/
│   ├── raw/
│   └── processed/
│       ├── 1_socioeconomics/
│       └── 2_analysis/
└── results/
```

- `code/` — scripts, **numbered** in the order they run
- `data/raw/` — read-only data from your source
- `data/processed/` — what your code produces
- `results/` — plots and tables for a paper or presentation
- `README.md` — what the project is and how to run it

The course folder follows the same convention.

Good setup – Project organization

- If you're going to rerun a script, you should **version** it
- Date-based folders is one option, `v1, ..., vn` another
- Keeps you from losing track of revisions

```
data/processed/2_analysis/  
├── 20260901/  
│   └── analysis_file.parquet  
└── 20260915/  
    └── analysis_file.parquet
```

Want a second opinion on folder structure? A similar convention, with slightly different names:

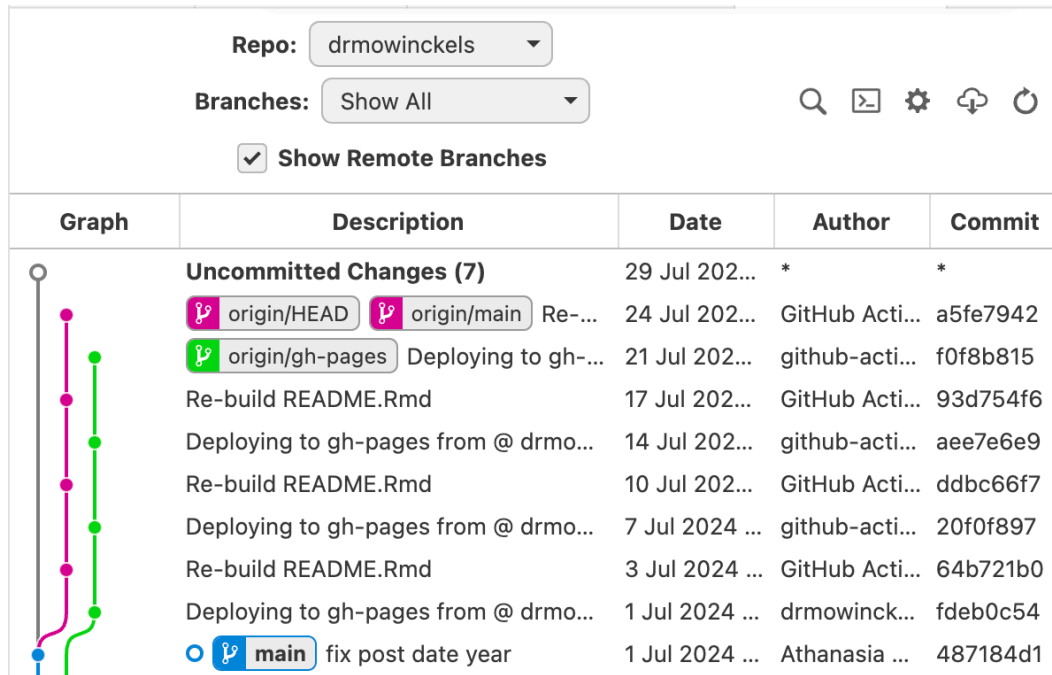
kdestasio.github.io/post/r_best_practices

Good setup – Git

Git is a **version control system** that tracks file changes as code develops.

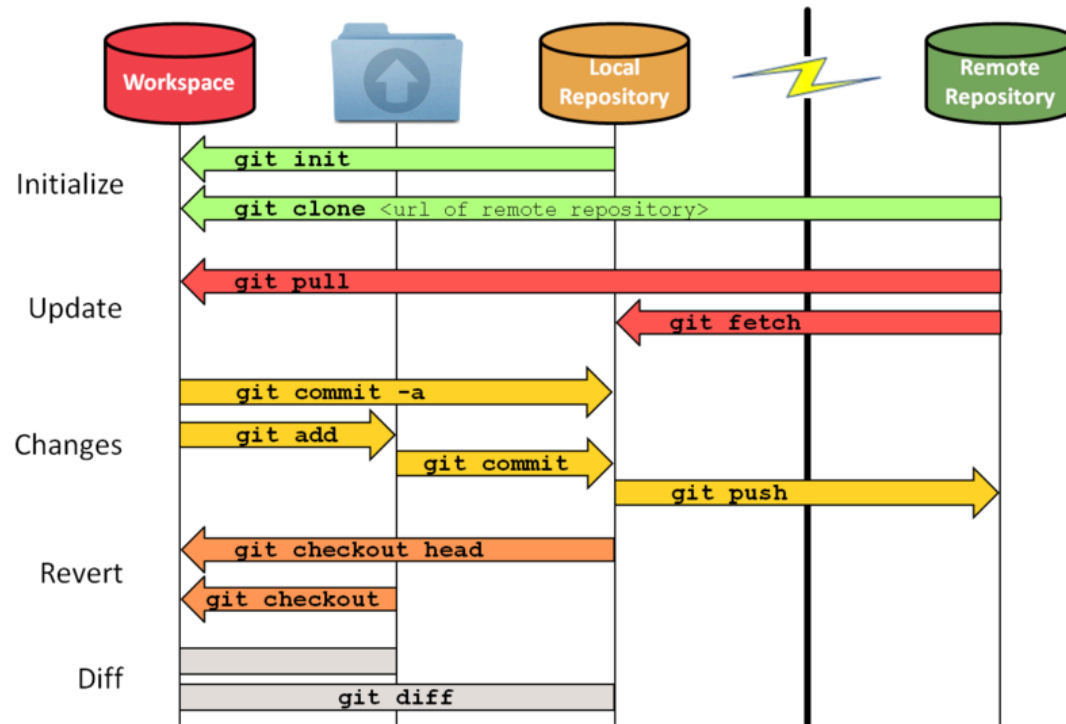
- Course materials distributed via Git
- Best practice in research and industry
- **Goal in this class:** get comfortable with Git for personal tracking
- Course covers basics, will develop over time

Good setup – Git: history over time



- **Commit:** an annotated snapshot at a specific point in time
- **Branch:** a stream of work that lets you experiment from current code or collaborate
- Main benefit is to see changes over time, go back in time when necessary

Good setup – Git: structure



- **Remote:** Published version of the repository
- **Local:** The version of the repo that Git tracks on your machine
- **Workspace** The folder on your computer where you make changes

Good setup – Air

Like writing, code is easier to understand and test if it follows a consistent grammar and structure. One way to build this into your R code is with **Air**.

“Raw” code

```
1 x=c(1,2,3)
2 y← mean(x)
3 if(y>0){
4     print( "positive" )
5 }
```

After Air runs

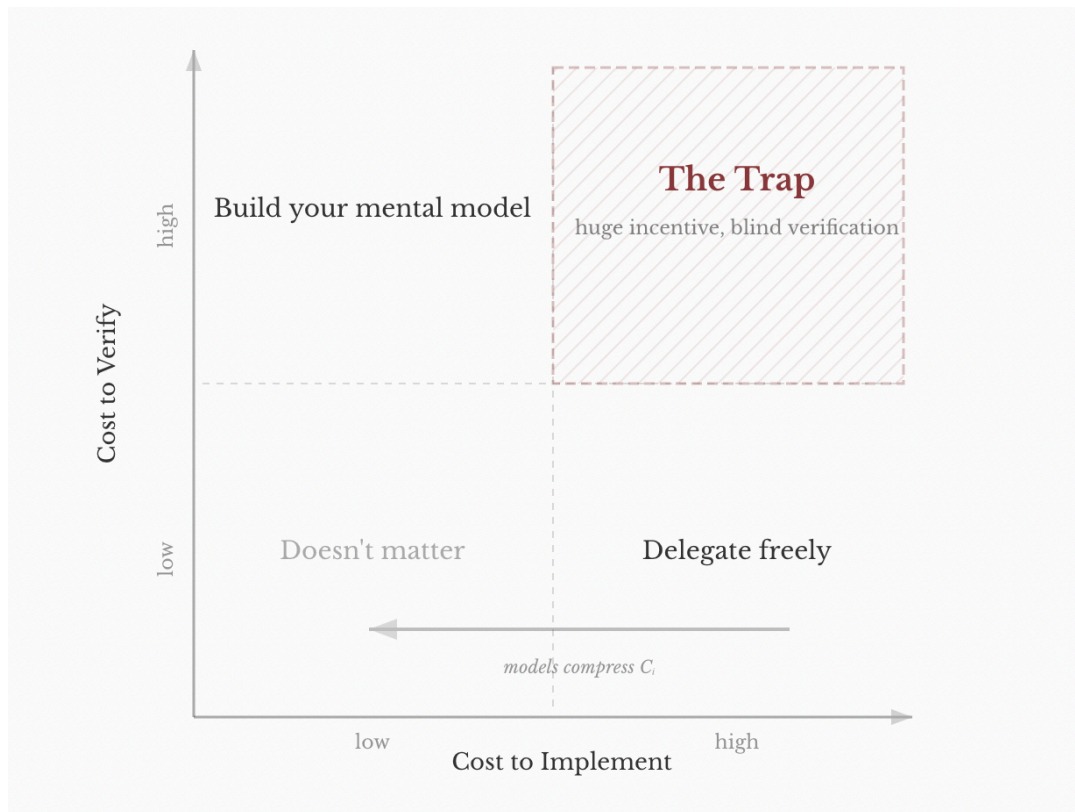
```
1 x ← c(1, 2, 3)
2 y ← mean(x)
3 if (y > 0) {
4     print("positive")
5 }
```

- Automatic spacing, indentation, and style
- Improves readability
- Set up to run **on save** in this course’s project

AI tools and coding

The new AI tools for coding may make it easier to generate code, but they make it **much harder** to verify.

This makes these skills even more important.



- Knowing what changes over time in your files
- Being able to see how data changes between operations
- Having clear documentation about how you transform your data

Resources and how to work with them

- Posit cheatsheets — dplyr, ggplot2, quarto, purrr
- R for Data Science (2e) — Wickham, Çetinkaya-Rundel & Grolemund
- Modern R Programming: Data Visualization — Kabacoff
- Fundamentals of Data Visualization — Wilke

**Next: Confirm your
setup and starting
the demo**

Before we start

- ☐ R installed
- ☐ Positron installed
- ☐ Git installed
- ☐ GitHub repository cloned
- ☐ Data downloaded from itslearning
- ☐ Demo file 1 open